

没问题！树（Tree）的算法题是面试和笔试中的“重灾区”，但好消息是它的**套路极强**。

只要你掌握了****递归（分治）和迭代（栈/队列）****这两种思维模式，90% 的树题都是下面这些模板的变体。

我为你整理了 **6 大核心树处理模板**，同样配上了详细的“保姆级”注释。

1. 树的三大遍历（递归版）

这是所有树算法的基石。记住：**前序是“传达指令”，后序是“汇报情况”**。

核心心法：根据你处理 `root.val` 的时机来决定用哪个。

适用场景：几乎所有树题目。

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def traverse(root):
    if not root: return

    # --- A. 前序位置 (Pre-order) ---
    # 时机：刚进入节点，还没去子节点。
    # 场景：打印目录结构、克隆树、前序序列化。
    # print(root.val)

    traverse(root.left)

    # --- B. 中序位置 (In-order) ---
    # 时机：从左子树回来，准备去右子树之前。
    # 场景：【★BST 二叉搜索树】专用，出来的顺序是有序数组。
    # print(root.val)

    traverse(root.right)

    # --- C. 后序位置 (Post-order) ---
    # 时机：左右子树都遍历完了，准备离开当前节点。
    # 场景：求树的高度、求节点数、删除树、LCA问题。
    # print(root.val)
```

2. 层序遍历 (BFS 模板)

核心心法：一次处理一行。

适用场景：求二叉树最大宽度、最小深度、右视图、锯齿形遍历。

```
import collections

def level_order(root):
    if not root: return []

    queue = collections.deque([root])
    results = []

    while queue:
        # 【★ 关键点】：锁死当前层的节点数
        # 这一行的长度必须在 for 循环前确定，因为循环里会加新节点
        level_size = len(queue)
        current_level_vals = []

        for _ in range(level_size):
            node = queue.popleft()
            current_level_vals.append(node.val)

            # 必须先左后右
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)

        results.append(current_level_vals)

    return results
```

3. 求树的属性 (自底向上 DFS)

核心心法：后序遍历 + 汇总信息。

适用场景：最大深度、判断平衡树、树的直径、最大路径和。

```
def get_tree_prop(root):  
    # 1. Base Case: 空节点, 高度为0, 或者贡献值为0  
    if not root:  
        return 0  
  
    # 2. 递归获取左右子树的信息 (Ask children)  
    left_h = get_tree_prop(root.left)  
    right_h = get_tree_prop(root.right)  
  
    # 3. 处理当前逻辑 (Process)  
    # 比如求最大深度: 左右最大者 + 自己(1)  
    current_h = max(left_h, right_h) + 1  
  
    # 比如判断是否平衡: 如果高度差 > 1, 标记为 -1 (代表不平衡)  
    # if abs(left_h - right_h) > 1: return -1  
  
    # 4. 向上汇报 (Return)  
    return current_h
```

4. 路径问题 (回溯法 Backtracking)

核心心法: 前序记录 + 到了叶子做判断 + 后序撤销。

适用场景: 路径总和 (Path Sum)、输出所有路径、从根到叶的所有数字。

```

def find_paths(root, target_sum):
    result = []
    path = []

    def backtrack(node, current_sum):
        if not node: return

        # --- 1. 做选择 (入栈) ---
        path.append(node.val)
        current_sum += node.val

        # --- 2. 判断是否到达叶子节点 ---
        # 只有在叶子节点才判断路径和是否符合要求
        if not node.left and not node.right:
            if current_sum == target_sum:
                result.append(path[:]) # 【★ 必须拷贝】

        # --- 3. 递归 ---
        backtrack(node.left, current_sum)
        backtrack(node.right, current_sum)

        # --- 4. 撤销选择 (出栈/回溯) ---
        # 离开当前节点前, 要把自己从路径里剔除, 不然会影响其他分支
        path.pop()

    backtrack(root, 0)
    return result

```

5. 最近公共祖先 (LCA)

核心心法：左右找人，如果左右都有，我就是；如果只有一边有，往上推。

适用场景：LeetCode 236 标准题、求两节点距离。

```

def lowestCommonAncestor(root, p, q):
    # 1. 终止条件
    # 如果到了空，或者找到了 p，或者找到了 q，直接返回当前节点
    # (这意味着告诉父节点："我这边发现了目标!")
    if not root or root == p or root == q:
        return root

    # 2. 去左右两边找
    left = lowestCommonAncestor(root.left, p, q)
    right = lowestCommonAncestor(root.right, p, q)

    # 3. 逻辑判断
    # 情况A: 左右两边都返回了非空值 -> 说明 p 和 q 分别在两边 -> root 就是 LCA
    if left and right:
        return root

    # 情况B: 只有一边找到了 -> 说明 p 和 q 都在同一侧 -> 返回找到的那一侧
    if left:
        return left
    if right:
        return right

    # 情况C: 左右都没找到
    return None

```

6. 二叉搜索树 (BST) 的验证与利用

核心心法：中序遍历是递增的 或 限定上下界 (Range)。

适用场景：验证 BST、BST 搜索、修剪 BST。

```

# 写法一：利用 Range（推荐，逻辑更严密）
def isValidBST(root):
    # 用 float('inf') 代表无穷大
    def validate(node, min_val, max_val):
        if not node: return True

        # 当前节点必须在 (min, max) 开区间内
        if not (min_val < node.val < max_val):
            return False

        # 往左走：上界变成当前值（左边的所有都要比当前小）
        # 往右走：下界变成当前值（右边的所有都要比当前大）
        return (validate(node.left, min_val, node.val) and
                validate(node.right, node.val, max_val))

    return validate(root, float('-inf'), float('inf'))

# 写法二：利用中序遍历（简单直观）
pre_val = float('-inf')
def inorder(node):
    if not node: return True
    if not inorder(node.left): return False
    if node.val <= pre_val: return False # 必须严格递增
    pre_val = node.val
    return inorder(node.right)

```

总结建议

1. 先看题目是“查表”还是“建房”：

- 如果是**只读**操作（求高度、找路径），套用模板 3 或 4。
- 如果是**修改/构造**操作（翻转树、构造树），通常用模板 1 的前序或后序位置来重新连接 `node.left = ...`。

2. BST 题目的作弊码：

- 一旦看到 **"二叉搜索树"** 这个词，马上想到 **"中序遍历有序"**，这通常是解题的最快捷径。

3. LCA 是分水岭：

- 如果你能完全理解模板 5 (LCA) 的逻辑，说明你对递归的返回值处理已经融会贯通了。
没问题！代码不仅要能跑，关键是隔了一个月再看还能**秒懂**。
我为你重新整理了这 5 个核心模板，并加上了**保姆级**的详细注释。这些注释解释了“为什么要这么写”以及“这里容易踩什么坑”。你可以直接复制保存到你的笔记软件里。

1. 万能回溯模板 (Backtracking)

核心心法：做选择 -> 递归 -> 撤销选择。

适用场景：全排列、组合、子集、分割回文串、棋盘问题（N皇后）。


```

def backtracking(start_index, current_path):
    """
    start_index: 本层递归开始遍历的位置（用于组合问题防止重复，全排列问题可能不需要它而需要 visited）
    current_path: 当前已经走过的路径（即已经做出的选择）
    """

    # --- 1. 终止条件 (Base Case) ---
    # 比如：路径长度达到要求，或者字符串切割完毕
    if 满足特定结束条件:
        # 【★ 重点坑位】：必须使用 [:] 进行切片拷贝！
        # 如果直接 result.append(current_path)，存进去的是内存引用。
        # 等你后面回溯把 path 删光了，result 里存的也会变为空列表。
        result.append(current_path[:])
        return

    # --- 2. 横向遍历（尝试当前层的所有可能选择） ---
    # range(start_index, len(choices)) 保证了我们不走回头路（针对组合问题）
    for i in range(start_index, len(choices)):
        choice = choices[i]

        # --- 3. 剪枝 (Pruning) (可选) ---
        # 如果当前选择明显会导致死胡同（比如和已经超过了目标值），直接跳过
        # if not is_valid(choice): continue

        # --- 4. 做选择 (Make Choice) ---
        # 就像下棋落子，或者把当前节点加入路径
        current_path.append(choice)

        # --- 5. 递归进入下一层 (Recursion) ---
        # 传入 i + 1 表示下一个数只能从后面选（不可重复选）
        # 如果题目允许重复选数字（如完全背包），这里可能传 i
        backtracking(i + 1, current_path)

        # --- 6. 撤销选择 (Backtrack / Undo) ---
        # 【★ 核心】：从下一层递归回来后，必须把刚才做的选择撤销掉。
        # 就像悔棋一样，把棋子拿起来，这样才能在下一轮循环尝试别的下法。
        current_path.pop()

```

2. 网格类 DFS 通用模板 (Grid DFS)

核心心法：越界检查 -> 标记访问 -> 扩散 -> (可选)还原。

适用场景：岛屿数量（沉岛）、单词搜索（迷宫）、最大连通面积。

```

def solve_grid(grid):
    rows, cols = len(grid), len(grid[0])

    # r, c 代表当前所在的行(row)和列(col)
    def dfs(r, c):
        # --- 1. 也是最重要的: 终止检查 (Stop Sign) ---
        # 顺序不能乱: 先检查越界, 再检查是否满足条件
        # 如果越界 (r, c 不在网格内)
        if not (0 <= r < rows and 0 <= c < cols):
            return False

        # 如果当前格子不是我们要找的目标, 或者已经被访问过('#')
        if grid[r][c] != '1':
            return False

        # --- 2. 标记已访问 (Mark Visited) ---
        # 为了防止递归回头造成死循环, 必须标记。
        # 方式A: 沉岛思想 (永久修改), 改为 '0' 或其他值, 适用于计算岛屿数量。
        # 方式B: 回溯思想 (临时修改), 存下旧值, 改为 '#', 适用于寻找路径/单词。
        temp = grid[r][c]
        grid[r][c] = '#' # 标记为已访问

        # --- 3. 向四个方向扩散 (Explore) ---
        # 上下左右递归, 只要有一条路通了就算通 (根据题目要求调整逻辑)
        found = (dfs(r + 1, c) or
                 dfs(r - 1, c) or
                 dfs(r, c + 1) or
                 dfs(r, c - 1))

        # --- 4. 还原现场 (Restore) (仅针对回溯类题目) ---
        # 如果题目是求“岛屿数量”, 不需要这一步 (淹了就淹了)。
        # 如果题目是“单词搜索”, 必须改回来, 因为别的路径可能还要用这个格子。
        # grid[r][c] = temp

        return found

    # 主循环: 遍历每一个格子作为起点
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1': # 找到入口
                dfs(r, c)

```

3. BFS 层序遍历模板 (Level-order BFS)

核心心法：队列 FIFO -> 记录当前层大小 -> 批量处理。

适用场景：二叉树层序遍历、最短路径步数、腐烂的橘子（多源BFS）。

```
import collections

def bfs(root):
    # 边界处理：如果是空树直接返回
    if not root: return []

    # --- 1. 初始化队列 ---
    # 使用 deque 是因为它的 popleft() 操作是 O(1) 的，而 list.pop(0) 是 O(n) 的
    queue = collections.deque([root])

    # 如果是图论 BFS（有环），这里必须还要有一个 visited = set([root])

    step = 0 # 记录层数/步数

    while queue:
        # --- 2. 获取当前层的节点数量 ---
        # 【★ 关键】：必须在 for 循环前锁死当前层的长度 level_size。
        # 因为在循环内部我们会 append 新节点，如果不锁死，循环就停不下来了。
        level_size = len(queue)

        # --- 3. 遍历当前这一层的所有节点 ---
        for _ in range(level_size):
            node = queue.popleft() # 拿出最老的节点

            # --- 4. 处理逻辑 ---
            # print(node.val)

            # --- 5. 拓展邻居 (Add Neighbors) ---
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)

        # 当前层处理完毕，步数 +1
        step += 1
```

4. 二叉树后序遍历/DFS 模板 (Bottom-up DFS)

核心心法：左右孩子先干活 -> 拿回结果 -> 父节点做决策。

适用场景：求树的高度、最大直径、判断平衡树、最近公共祖先 (LCA)。

```
def dfs(node):
    # --- 1. 终止条件 (Base Case) ---
    # 到底了 (空节点), 返回基础值
    # 求高度返回 0, 求节点返回 None, 根据题目变通
    if not node:
        return 0

    # --- 2. 递归获取子节点信息 (Delegate) ---
    # 就像老板问下属, 必须等下属有了结果, 老板才能做决定
    left_info = dfs(node.left) # 左边的结果
    right_info = dfs(node.right) # 右边的结果

    # --- 3. 处理当前节点逻辑 (Process) ---
    # 结合左右孩子的信息, 计算当前节点的信息

    # 案例: 计算树的高度
    current_height = max(left_info, right_info) + 1

    # 案例: 寻找最近公共祖先 LCA
    # if left_info and right_info: return node (找到了分叉点)

    # --- 4. 向上层汇报 (Return) ---
    return current_height
```

5. 复杂输入处理模板 (ACM 模式)

核心心法：读完所有字符 -> 变成迭代器 -> 像吃豆子一样一个个吐出来。

适用场景：输入包含多行、空格数量不固定、不知道具体有多少个数据 (需要 try-except)。

```

import sys

# 【★ 防爆设置】：Python 默认递归深度只有 1000，刷题时（特别是树和图）很容易爆栈。
# 设置大一点（比如 5000 或 100000）保平安。
sys.setrecursionlimit(5000)

def solve():
    # --- 1. 一次性读取 ---
    # sys.stdin.read() 会把输入流中的所有内容（包括换行）读成一个长字符串。
    # .split() 默认按所有空白字符（空格、换行、制表符）分割，自动去除空字符串。
    # 这是一个处理 "烂输入" 的神器。
    raw_data = sys.stdin.read().split()

    if not raw_data: return

    # --- 2. 制作迭代器 ---
    # iter() 把列表变成迭代器，这样我们可以用 next() 一个个取值，
    # 而不需要手动维护一个 index 变量（比如 data[i]）。
    iterator = iter(raw_data)

    try:
        while True:
            # --- 3. 消费数据 ---
            # 尝试拿下一个 token
            n_str = next(iterator)
            n = int(n_str)

            # 根据 n 再去拿 n 个数据或者 n 行数据
            # for _ in range(n):
            #     val = int(next(iterator))
            #     ... 处理逻辑 ...

            # 打印当前测试用例的结果
            # print(result)

    except StopIteration:
        # --- 4. 结束处理 ---
        # 当 next() 拿不到数据时，会抛出 StopIteration，代表输入结束。
        pass

if __name__ == '__main__':

```

solve()

祝你考试顺利！这对你来说是一个关键时刻。根据我们之前的交流，我为你整理了一份**“考前急救包”

你练习的题目主要集中在 **DFS/回溯**、**树的各种操作（构建、遍历、属性）**、**二分查找** 以及 **字符串/输入处理**。

以下是知识点全景图和必须死记硬背的代码框架（Python 版）。

一、知识点全景图 (Mind Map)

1. 搜索与回溯 (DFS/Backtracking)

- **核心逻辑**：不撞南墙不回头。
- **应用**：网格搜索（单词搜索）、路径寻找、全排列/组合。
- **关键点**：状态标记（visited）、递归深入、**回溯（撤销标记）**。

2. 树 (Trees) - 重中之重

- **本质**：递归结构，链式存储。
- **遍历**：前序（根左右）、中序（左根右）、后序（左右根）、层序（BFS）。
- **构建**：
 - 前序+中序 后序。
 - 带度数的序列 树（用 Queue）。
 - 括号嵌套字符串 树（用 Stack）。
- **特殊树**：二叉搜索树（BST，中序有序）、完全二叉树（利用索引）。

3. 二分查找 (Binary Search)

- **核心逻辑**：减治法，每次排除一半。
- **应用**：有序数组找数、**二分答案**（切木头，求最大化最小值）。

4. 数据结构辅助

- **Stack**：处理嵌套结构（括号、目录层级）。
- **Queue**：处理层序遍历、广度优先搜索。
- **Set/Hash**：处理去重、快速查找（点名签到）。

二、必背算法模板 (Code Templates)

1. 网格 DFS & 回溯模板 (Grid DFS)

适用题目：单词搜索、岛屿数量、迷宫路径。

```
# 假设 grid 是二维网格
rows, cols = len(grid), len(grid[0])

def dfs(r, c, visited):
    # 1. 越界检查 (Base Case 1)
    if not (0 <= r < rows and 0 <= c < cols):
        return False

    # 2. 有效性检查 (Base Case 2)
    # 比如：遇到了障碍物，或者已经访问过，或者字符不匹配
    if grid[r][c] == '障碍' or visited[r][c]:
        return False

    # 3. 成功终止条件 (Base Case 3)
    if 满足目标条件: return True

    # 4. 做选择 (Mark)
    visited[r][c] = True # 或者 grid[r][c] = '#'

    # 5. 递归扩散 (上下左右)
    # 只要有一条路通就可以
    res = (dfs(r+1, c, visited) or
           dfs(r-1, c, visited) or
           dfs(r, c+1, visited) or
           dfs(r, c-1, visited))

    # 6. 撤销选择 (Backtrack) - 回溯的关键！
    # 如果是找“岛屿数量”这种淹没模型，不需要这步
    # 如果是“寻找路径”或“单词搜索”，必须还原
    visited[r][c] = False

    return res
```

2. 树的递归与重构模板 (Tree Recursion)

适用题目：求树高、路径和、前序+中序重建树、文件目录解析。


```

class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

# 通用递归框架（以 前序+中序 -> 后序 为例）
def build_tree(preorder, inorder):
    if not preorder: return None

    # 1. 找到根节点
    root_val = preorder[0]
    root = TreeNode(root_val)

    # 2. 切分左右子树 (Find Split Point)
    mid_idx = inorder.index(root_val)

    # 左子树的长度
    left_len = mid_idx

    # 3. 递归构建 (Divide and Conquer)
    # 注意索引范围，这是最容易错的地方
    root.left = build_tree(preorder[1 : 1+left_len], inorder[:mid_idx])
    root.right = build_tree(preorder[1+left_len :], inorder[mid_idx+1:])

    return root

```

3. BFS / 层序遍历模板 (Queue)

适用题目：树的层序遍历、最短路径步数、带度数的序列还原树。

```
import collections

def bfs(root):
    if not root: return

    queue = collections.deque([root])

    while queue:
        # 【重要】锁定当前层的大小
        level_size = len(queue)

        # 遍历当前层的所有节点
        for _ in range(level_size):
            node = queue.popleft()

            # 处理当前节点
            # print(node.val)

            # 将下一层加入队列
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
            # 如果是 N 叉树: for child in node.children: queue.append(child)
```

4. 二分答案模板 (Binary Search on Answer)

适用题目：切木头、分巧克力、最大化最小值。

```

def check(mid):
    # 根据题意编写检查逻辑
    # 如果以 mid 为标准, 是否能满足题目要求 (比如能切出 K 段)
    count = 0
    for x in data:
        count += x // mid
    return count >= K

def solve():
    left, right = 1, max_value # 确定搜索范围
    ans = 0

    while left <= right:
        mid = (left + right) // 2
        if mid == 0: # 防除0
            left += 1
            continue

        if check(mid):
            ans = mid # 记录可行解
            left = mid + 1 # 贪心: 尝试更大的
        else:
            right = mid - 1 # 不可行, 尝试更小的
    return ans

```

三、 考场必备：输入输出处理 (ACM 模式)

你的很多题目需要自己处理复杂的输入（多行、未知长度），**这个模板请务必背熟**，能解决 90% 的输入读取问题。

```

import sys

# 【防爆栈】树的题目如果太深，Python默认深度不够，手动加满
sys.setrecursionlimit(5000)

def solve():
    # 1. 一次性读取所有 token，自动处理换行和空格
    input_data = sys.stdin.read().split()

    if not input_data: return

    # 2. 制作迭代器，像吃豆子一样一个个拿
    iterator = iter(input_data)

    try:
        while True:
            # 3. 根据题目逻辑读取
            # 比如先读 N
            n_str = next(iterator)
            n = int(n_str)

            # 再读 N 个数
            # nums = []
            # for _ in range(n):
            #     nums.append(int(next(iterator)))

            # 处理逻辑...
            # print(result)

    except StopIteration:
        pass # 读取完毕

if __name__ == '__main__':
    solve()

```

四、考前叮嘱 (Tips)

1. 看清数据范围：

- ：可以用 的暴力回溯。
- ：可以用 。
- ：必须用 (排序/二分) 或 。

- ：必须用 (完全二叉树索引计算) 或。

2. 变量初始化：

- 多组测试数据时，**全局变量**（如 `result_list` , `visited`）一定要在 `while` 循环内部或每组开始时**清空/重置**。

3. 边界条件：

- 二分查找 `mid=0` 怎么办？
- 树是空的 (`root is None`) 怎么办？
- 数组越界 (`i < 0` or `i >= len`) 检查了吗？

考试时不要慌，遇到树就画图，遇到递归就想“甩锅”给子函数。相信你这段时间的练习，加油！

这对你应对即将到来的考试至关重要。在算法笔试（尤其是 ACM/OJ 模式）中，**输入输出 (I/O)** 往往是**第一道坎**。如果数据读不对，算法再精妙也是 0 分。

Python 的 I/O 其实就分两派：“**教科书派**” (`input`) 和 “**实战竞赛派**” (`sys.stdin`)。

以下是详细总结，建议根据题目类型选择最合适的方法。

一、 输入篇 (Input)

1. 基础写法： `input()`

适用场景：数据量小、交互式简单题目、LeetCode 某些特定题目（通常 LeetCode 不用处理输入，但部分公司笔试也是 LeetCode 模式却需要自己写 I/O）。

- **读一行字符串：**

```
s = input() # 读入 "hello world"
```

- **读一个整数：**

```
n = int(input()) # 读入 "10" -> 10
```

- **读一行空格分隔的整数（最常用）：**

```
# 输入: "1 2 3 4 5"
nums = list(map(int, input().split()))
# 结果: [1, 2, 3, 4, 5]
```

- 读多行固定数据:

```
n = int(input()) # 先读行数
lines = []
for _ in range(n):
    lines.append(input())
```

缺点: 速度慢。当输入行数超过 或 时, `input()` 可能会导致 **TLE (超时)**。

2. 进阶写法: `sys.stdin` (笔试必备)

适用场景: 所有算法竞赛、数据量大、多行输入、不确定行数、包含复杂空格换行的数据。

首先引入库: `import sys`

方法 A: `sys.stdin.readline()` (比 `input` 快)

用法和 `input()` 几乎一样, 但更快。

注意: 它会把行末的换行符 `\n` 也读进去, 所以通常要配合 `.strip()`。

```
import sys
# 读取单行
line = sys.stdin.readline().strip()
# 读取数字
n = int(sys.stdin.readline())
```

方法 B: `sys.stdin.read().split()` (神技: 流式处理) ✨

这是我最推荐你掌握的方法, 也就是之前带你写的 `iterator` 写法。

它的原理是: 不管有多少行、多少空格、多少回车, 一口气全读进来, 变成一个巨大的单词列表。

模板代码:

```
import sys

def solve():
    # 1. 一次性吞掉所有输入
    data = sys.stdin.read().split()
    if not data: return

    # 2. 变成迭代器，按顺序“取号”
    iterator = iter(data)

    try:
        while True:
            # 3. 需要什么就拿什么
            # 比如题目说：先给一个 N，再给 N 个数
            n = int(next(iterator))

            nums = []
            for _ in range(n):
                val = int(next(iterator))
                nums.append(val)

            # 处理逻辑...
            # print(ans)

    except StopIteration:
        # 数据读完了，自动退出
        pass

if __name__ == '__main__':
    solve()
```

优点：

1. **极快**：Python 中最快的读取方式。
2. **无视格式**：不用管是一行一个数，还是一行多个数，或者是空行，`split()` 会自动过滤所有空白。
3. **处理 EOF**：配合 `try-except` 完美解决“读到文件结束”的问题。

二、 输出篇 (Output)

1. 基础输出： print()

- 默认换行： `print(a)` 会自动在末尾加 `\n`。
- 不换行： `print(a, end="")` 或 `print(a, end=" ")`（用空格结尾）。

2. 格式化输出 (保留小数)

考试常考“保留两位小数”。

- **f-string (推荐):**

```
pi = 3.1415926
print(f"{pi:.2f}") # 输出 3.14
```

- **format 方法:**

```
print("{:.2f}".format(pi))
```

3. 列表输出 (Array Printing)

题目常要求：输出一行数字，用空格隔开，行末不能有空格。

- **错误写法** (行末多空格，有时会判错):

```
for x in nums:
    print(x, end=" ")
```

- **正确写法** (`join` 大法):

```
# 必须先把数字转成字符串 str(x)
print(" ".join(map(str, nums)))
```


三、典型场景实战代码

为了让你考试时不慌，我总结了 3 种最常见的输入场景。

场景 1：第一行 N，后面跟着 N 行数据

输入：

3

A 10

B 20

C 30

```
import sys
input = sys.stdin.readline # 替换 input 函数，加速

n = int(input())
for _ in range(n):
    # line 可能是 "A 10"
    parts = input().split()
    name = parts[0]
    score = int(parts[1])
    # 处理...
```

场景 2：多组测试数据，直到 EOF (End of File)

输入：

1 5

10 20

... (不知道多少行，读完为止)

```
import sys

# 写法 A: 逐行读
for line in sys.stdin:
    # line 就是读到的一行
    if not line.strip(): break
    a, b = map(int, line.split())
    print(a + b)

# 写法 B: 迭代器法 (推荐, 逻辑更连贯)
data = sys.stdin.read().split()
iterator = iter(data)
try:
    while True:
        a = int(next(iterator))
        b = int(next(iterator))
        print(a + b)
except StopIteration:
    pass
```

场景 3: 二维网格 (Matrix) 输入

输入:

```
3 3 (行 列)
0 1 0
1 1 1
0 0 0
```

```
import sys
input = sys.stdin.readline

rows, cols = map(int, input().split())
grid = []
for _ in range(rows):
    # 读一行, 转成整数列表, 加入 grid
    row = list(map(int, input().split()))
    grid.append(row)

# 此时 grid[0][1] 就是第一行第二列的数
```

四、考前最后检查清单 (Checklist)

1. **一定要 Import**: `import sys`, `import collections` (如果用 `Queue`), `sys.setrecursionlimit(5000)` (如果用 `DFS`)。
2. **输入是字符串**: 切记 `input()` 或 `next()` 拿到的都是 `str`, 做计算前**千万别忘了 `int()`**。
3. **切片要拷贝**: 回溯存结果时, 记得用 `path[:]` 或 `list(path)`。
4. **变量重置**: 如果有多组数据 (`while True`), 记得在每组开始前把 `ans`, `visited` 等变量**清零**。

把那个 `sys.stdin.read().split() + iterator**` 的模板背下来, 它能解决你遇到的大部分“输入格式恶心”的问题。加油!

图论 (Graph) 是算法考试中最容易拉开分差的板块。它的难点在于模型识别**：很多题目表面是在问“植物分类”、“修路”、“选课”，其实背后对应着标准的图论算法。

只要你掌握了**建图**和**4 个核心模板**, 就能应付绝大多数笔试题。

一、图的存储 (建图)

在 Python 中, 我们极少用“邻接矩阵”(二维数组), 除非节点数。
最通用的是 **邻接表 (Adjacency List)**。

通用建图模板:

```
import collections

# 假设输入是 m 条边: u v w (u到v有一条权重为w的边)
graph = collections.defaultdict(list)

for _ in range(m):
    u, v, w = map(int, input().split())

    # 1. 有向图 (Directed)
    graph[u].append((v, w))

    # 2. 无向图 (Undirected) - 记得双向添加!
    graph[v].append((u, w))
```

二、核心算法模板

1. 并查集 (Union-Find) —— 图的连通性

必考指数：★★★★★

适用场景：判断两个点是否连通、朋友圈数量、判断图中是否有环、最小生成树 (Kruskal)。

```
class UnionFind:
    def __init__(self, n):
        # 初始化：每个人的父亲是自己
        self.parent = list(range(n + 1))

    def find(self, x):
        # 路径压缩：查找时顺便把沿途节点的父节点都直接指向根
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        rootX = self.find(x)
        rootY = self.find(y)
        if rootX != rootY:
            self.parent[rootX] = rootY
            return True # 合并成功
        return False # 已经在同一集合，无需合并（或者说明有环）

# 使用示例：
# uf = UnionFind(n)
# if uf.union(u, v): ...
```

2. 最短路径：Dijkstra 算法

必考指数：★★★★★

适用场景：带权重的最短路径（如：地图导航、网络延迟）。注意：边的权重必须非负。

核心逻辑：贪心 + 优先队列（堆）。

```
import heapq

def dijkstra(start_node, n, graph):
    # 1. 初始化距离表，无穷大
    dist = [float('inf')] * (n + 1)
    dist[start_node] = 0

    # 2. 优先队列：存 (距离, 节点)，按距离从小到大自动排序
    pq = [(0, start_node)]

    while pq:
        d, curr = heapq.heappop(pq)

        # 3. 剪枝：如果当前取出的距离比已记录的距离大，说明是过期信息
        if d > dist[curr]:
            continue

        # 4. 遍历邻居
        for neighbor, weight in graph[curr]:
            new_dist = d + weight

            # 5. 松弛操作 (Relaxation)
            if new_dist < dist[neighbor]:
                dist[neighbor] = new_dist
                heapq.heappush(pq, (new_dist, neighbor))

    return dist # 返回从起点到所有点的最短距离
```

3. 拓扑排序 (Topological Sort)

必考指数：★★★★

适用场景：课程表问题（先修课）、任务依赖顺序、判断有向图是否有环。

核心逻辑：入度表 (Indegree) + 队列。

```

def topological_sort(n, edges):
    # 1. 建图 + 统计入度
    graph = collections.defaultdict(list)
    indegree = [0] * n

    for u, v in edges:
        graph[u].append(v)
        indegree[v] += 1 # v 有一个依赖 u

    # 2. 将所有入度为 0 的节点（没有依赖）入队
    queue = collections.deque([i for i in range(n) if indegree[i] == 0])
    result = []

    while queue:
        curr = queue.popleft()
        result.append(curr)

        # 3. 削减邻居的入度
        for neighbor in graph[curr]:
            indegree[neighbor] -= 1
            # 如果入度变成 0，说明依赖解除了，可以入队
            if indegree[neighbor] == 0:
                queue.append(neighbor)

    # 4. 判断是否有环
    if len(result) == n:
        return result # 返回排序结果
    else:
        return [] # 有环，无法完成排序

```

4. 多源最短路：Floyd 算法 (Floyd-Warshall)

必考指数：☆☆☆

适用场景：求任意两点之间的最短路，数据量很小 ()。

核心逻辑：三层循环暴力更新。

```
# 初始化: graph[i][j] = 权重, 如果不通则为 inf, graph[i][i] = 0
def floyd(n, graph):
    # dp[k][i][j] 的空间优化版
    for k in range(n): # 中转点 k
        for i in range(n): # 起点 i
            for j in range(n): # 终点 j
                # 如果 i->k->j 比 i->j 更近, 更新
                graph[i][j] = min(graph[i][j], graph[i][k] + graph[k][j])
```

三、考场图论决策树 (如何选算法?)

看到题目后, 按以下顺序思考:

1. 是连通性问题吗? (问分几组、是否连通)
 - 并查集 (Union-Find) (最快) 或 BFS/DFS。
2. 是有依赖关系的排序吗? (选课、工序)
 - 拓扑排序 (Kahn算法)。
3. 是最短路径吗?
 - 没有权重 (每步代价都一样) **BFS**。
 - 有权重 (非负) **Dijkstra** (首选)。
 - 有负权 Bellman-Ford / SPFA (考得少)。
 - 任意两点距离 & N很小 **Floyd**。
4. 是最小生成树吗? (修路连通所有城市, 造价最低)
 - **Kruskal 算法** (本质是: 按边权排序 + 并查集)。

四、考前避坑指南

1. 节点编号陷阱:
 - 题目给的节点是 1 到 N, 还是 0 到 N-1?
 - 如果是 1 到 N, 建议开数组时开 N + 1 大小, 不用手动减 1, 防止弄混。
2. 无向图陷阱:
 - 如果是“公路”、“朋友关系”, 通常是无向的。

- **建图时必须 $u \rightarrow v$ 和 $v \rightarrow u$ 都加！** 否则 Dijkstra 或 BFS 跑不通。

3. 无穷大：

- Python 用 `float('inf')` 表示无穷大。千万别用 99999，万一路径和超过它就挂了。
把这几个模板（尤其是 **并查集** 和 **Dijkstra**）背下来，图论题通常就是套模板改改输入。加油！